

CuFDFB: Fast and Private Computation on Non-Linear Functions Using FHE

Shutong Jin, *Student Member, IEEE*, Shiyu Shen, *Member, IEEE*, Hao Yang, Donglong Chen *Member, IEEE*, Wangchen Dai and Ray C. C. Cheung*, *Senior Member, IEEE*

Abstract—Privacy-preserving neural network inference using Fully Homomorphic Encryption (FHE) faces significant challenges in efficiently evaluating non-polynomial functions, such as activation functions, which are critical for introducing non-linearity in neural networks. Full-Domain Functional Bootstrap (FDFB) algorithms provide a promising solution by enabling the evaluation of arbitrary functions while simultaneously refreshing ciphertexts to manage noise accumulation. Despite their theoretical advantages, the practicality of FDFB algorithms has been limited by excessive computational overhead, often exceeding 1000 ms per ciphertext, which restricts their scalability for large neural networks.

To overcome the computational bottlenecks of FDFB, we have re-engineered the algorithms for massively parallel execution on GPUs. Our primary contribution is a hierarchical parallelization strategy that exploits concurrency at the thread, stream, and device levels. A key optimization involves the use of CUDA streams to create a data pipeline that effectively mitigates the overhead of memory transfers between the host and device. This optimized architecture achieves a significant speedup of up to $524\times$ compared to CPU-based implementations. Our implementation maintains full precision for evaluating various activation functions, confirming its viability for large-scale, privacy-preserving machine learning tasks and paving the way for practical FHE-based deep learning.

Index Terms—Fully Homomorphic Encryption, Full-Domain Functional Bootstrapping, TFHE, GPU Acceleration

I. INTRODUCTION

The increasing adoption of machine learning in sensitive domains such as healthcare, finance, and national security has raised significant concerns regarding data privacy and security. These concerns necessitate the development of Privacy-Preserving Machine Learning techniques, which allow computations to be performed on encrypted data without compromising sensitive information. Securing neural network inference is particularly crucial, as it ensures the privacy of both the data-provider and the model-provider.

Secure neural network inference enables computations on encrypted data without exposing either the input data or the model parameters. This ensures that the client's private data remains confidential from the server, and the server's trained

model is protected from potential misuse by the client [1], [2]. Various cryptographic techniques have been employed to achieve privacy-preserving inference, including Fully Homomorphic Encryption (FHE) [1], [2], Secret Sharing (SS) [3]–[5], Garbled Circuits (GC) [6], [7], and Oblivious Transfer (OT) [8]–[10].

CryptoNets [1] introduced the first FHE-based secure inference framework, demonstrating how neural networks could be evaluated on encrypted data. Subsequent advancements have integrated multiple cryptographic primitives to improve computational efficiency and accuracy. For example, linear operations like matrix multiplications are efficiently evaluated using FHE and SS, while non-linear functions are typically handled using GC or OT [2], [9]. Specialized schemes such as DeepSecure [2] and XONN [9] have optimized secure inference protocols for binary neural networks. Additionally, frameworks like Swift [11] combine FHE and SS to enable low-latency computations on encrypted data.

Among cryptographic solutions, FHE stands out for its ability to perform arbitrary computations directly on encrypted data. FHE eliminates the need for decryption, offering a robust solution for secure and privacy-preserving neural network inference. Schemes such as CKKS [12] support approximate arithmetic on real numbers, making them suitable for evaluating linear operations prevalent in neural networks. Efficiency is further enhanced by techniques such as Single Instruction Multiple Data (SIMD) packing, which encodes multiple plaintexts into a single ciphertext for parallel processing. Another key advantage of FHE is its extensibility, as it provides a universal API for homomorphic addition and multiplication. This extensibility has enabled the development of FHE compilers [13], [14], making FHE more accessible to security developers.

FHE schemes are broadly categorized into *word-type FHE* and *boolean-type FHE*, each addressing different computational requirements and trade-offs. *Word-type FHE* schemes, such as BGV [15], BFV [16]–[18], and CKKS [12], are optimized for large plaintext spaces and support efficient data packing through techniques like SIMD, making them well-suited for high-throughput applications. However, they face significant challenges in handling non-linear functions due to the high computational costs associated with bootstrapping and key-switching, which introduce considerable overhead. To manage these limitations, low-degree polynomial approximations, such as degree-2 polynomials for ReLU in CryptoNets [19], are often employed. While such approximations are computationally efficient, they lack the precision required for modern neural networks, which often feature complex

Shutong Jin, Shiyu Shen and Ray C. C. Cheung are with the Department of Electrical Engineering, City University of Hong Kong (e-mail: shutong.jin@my.cityu.edu.hk; crypto@sherie.dev; r.cheung@cityu.edu.hk).

Hao Yang is with the Department of Data Science, City University of Hong Kong (e-mail: crypto@d4rk.dev)

Donglong Chen is with Guangdong Provincial/Zhuhai Key Laboratory of IRADS, Beijing Normal-Hong Kong Baptist University, China (e-mail: donglongchen@uic.edu.cn).

Wangchen Dai is with School of Cyber Science and Technology, Sun Yat-Sen University, China (e-mail: daiwch@mail.sysu.edu.cn).

architectures and advanced activation functions like GELU, used in state-of-the-art models such as BERT [20] and GPT-3 [21].

Boolean-type FHE schemes, such as FHEW [22] and TFHE [23], [24], build upon the ciphertext structure introduced by Gentry et al. [25], address these challenges by supporting efficient evaluation of boolean gates and lookup tables through programmable bootstrapping. Full Domain Functional Bootstrap (FDFB) algorithms, are specifically designed for evaluating non-linear functions. By leveraging Look-Up Tables (LUTs) during bootstrapping, FDFB enables accurate evaluations of arbitrary unary functions, such as activation functions, across the full input domain [23], [24]. This dual functionality—noise reduction and function evaluation—makes FDFB a promising approach for implementing non-linear functions in FHE-enabled neural networks.

Despite its theoretical advantages, FDFB faces practical challenges, particularly in terms of computational demands. The high computational cost of FDFB, especially for large-scale neural networks, limits its real-world applicability. Current state-of-the-art implementations on CPUs exhibit evaluation times exceeding 1000 ms per ciphertext [26]–[29], which is impractical for large-scale neural network inference.

The parallel processing capabilities of GPUs offer a promising solution to these computational challenges. GPU acceleration techniques are widely used in post-quantum cryptography [30], [31] and shown to be effective in reducing the computational overhead of FHE operations [32]–[35], enabling faster evaluations of complex functions and improving the overall performance of secure inference frameworks.

This work presents the first GPU-accelerated implementation of FDFB algorithms, bridging the gap between theoretical advancements and real-world applicability.

Major Contributions

The major contributions of this work are:

- 1) We bridged the gap between theoretical research on FDFB algorithms and their practical applicability by developing a GPU-accelerated implementation for a diverse range of FDFB algorithms. To the best of our knowledge, this work represents the first GPU-accelerated implementation in this field, marking a significant step toward enhancing the real-world feasibility and performance of FDFB algorithms. Our implementation adopts a structured, multi-level concurrency model, effectively exploiting parallelism at the thread, stream, and CPU-GPU collaboration levels. These optimizations deliver a significant performance boost, achieving a speedup of up to **524 times** compared to equivalent CPU-based implementations.
- 2) We perform a comprehensive performance evaluation of our GPU-accelerated FDFB algorithms, with an emphasis on their applicability to various activation functions commonly used in neural networks. The experimental results demonstrate that our implementation supports efficient and consistent evaluation for a wide range of activation functions, irrespective of their continuity or

differentiability. Notably, this capability is achieved with an evaluation time of less than 1 ms per ciphertext, highlighting the practicality and speed of the proposed approach.

- 3) We point out the limitations of previous analyses of FDFB algorithms, as they only provide a rough estimation of the number of blind rotations involved, which is not necessarily in line with computational overhead. Specifically, we analyzed the performance degradation observed in FDFB-Select experiments, identifying the real impact of operations such as Number Theoretic Transforms (NTTs) and Hadamard products on computational efficiency. This in-depth examination explains why certain algorithms, like FDFB-Select, do not perform optimally in GPU implementations and provides a foundation for future improvements.
- 4) We present a thorough evaluation of FHE-enabled secure inference for non-linear functions, encompassing both LUT-based approaches and polynomial-approximation techniques. Our results demonstrate the practicality and robustness of FDFB-powered algorithms in secure neural network inference scenarios. The analysis underscores the strengths of FDFB in accurately and efficiently processing non-polynomial activation functions, highlighting its potential to bridge the gap between privacy-preserving computation and the scalability demands of modern deep learning applications..

II. PRELIMINARIES

A. Notations

The Learning with Errors (LWE) problem is defined over $\mathbb{Z}_q$. An LWE sample is represented as $(\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, where $\mathbf{a} \in \mathbb{Z}_q^n$ is chosen uniformly at random, $b = \langle \mathbf{a}, \mathbf{s} \rangle + e \mod q$, $\mathbf{s} \in \mathbb{Z}_q^n$ is the secret key, and $e \in \mathbb{Z}$ is sampled from a discrete Gaussian distribution χ . The decisional LWE problem is to distinguish between such samples and uniformly random pairs $(\mathbf{a}, b)$.

The Ring Learning with Errors (RLWE) problem generalizes LWE to polynomial rings. In RLWE, samples are of the form $(\mathbf{a}, b) \in \mathcal{R}_q \times \mathcal{R}_q$, where $\mathbf{a} \in \mathcal{R}_q$ is uniformly random, and $b = \mathbf{a} \cdot \mathbf{s} + e \mod (X^N + 1)$, with $\mathbf{s}, e \in \mathcal{R}_q$, and e sampled from a Gaussian distribution over the coefficients. The decisional RLWE problem is to distinguish such samples from uniformly random pairs in $\mathcal{R}_q \times \mathcal{R}_q$.

TABLE I: List of symbols.

Symbol	Description
p	Plaintext modulus
q	Ciphertext modulus
n	Dimension of LWE ciphertext
Q	Modulus for RLWE ciphertext
N	Dimension of RLWE ciphertext
q_{ks}	Modulus for key switching
q_{pk}	Modulus for packed key switching
B_q	Gadget decomposition base for bootstrapping
B_{ks}	Gadget decomposition base for key switching
B_{pk}	Gadget decomposition base for packed key-switching

B. Ciphertext Construction

1) *LWE Ciphertexts*: The LWE ciphertext is defined as a pair $(\mathbf{a}, b)$ over $\mathbb{Z}_q^{n+1}$, where:

$$\text{LWE}_{\mathbf{s},q}(m) = \left(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + \left\lfloor \frac{q}{t} \cdot m \right\rfloor + e \right),$$

with the following parameters:

- $\mathbf{a} \in \mathbb{Z}_q^n$: a uniformly random vector,
- $b \in \mathbb{Z}_q$: the ciphertext component,
- $\mathbf{s} \in \mathbb{Z}_q^n$: the secret key vector,
- $m \in \mathbb{Z}_t$: the plaintext message,
- $e \in \mathbb{Z}_q$: a small error term sampled from a discrete error distribution.

The decryption process recovers the plaintext m as:

$$\text{LWE}_{\mathbf{s},q}^{-1}(\mathbf{a}, b) = \left\lfloor \frac{t}{q} \cdot [b - \langle \mathbf{a}, \mathbf{s} \rangle]_q \right\rfloor_t$$

2) *RLWE Ciphertexts*: The RLWE ciphertext is an extension of LWE to polynomial rings. It is defined as:

$$\text{RLWE}_{\mathbf{s},Q}(m) = \left(\mathbf{a}, \mathbf{b} = \mathbf{a} \cdot \mathbf{s} + \left\lfloor \frac{Q}{t} \cdot m \right\rfloor + e \right),$$

with the following parameters:

- $\mathbf{a} \in \mathcal{R}_Q$: a uniformly random polynomial over the ring $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R}$ (where $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ is the $2N$ -th cyclotomic polynomial ring),
- $\mathbf{b} \in \mathcal{R}_Q$: the ciphertext component,
- $\mathbf{s} \in \mathcal{R}_Q$: the secret key polynomial,
- $m \in \mathcal{R}_p$: the plaintext message encoded as a polynomial,
- $e \in \mathcal{R}_Q$: a small error polynomial sampled from a discrete Gaussian distribution.

The plaintext m is recovered during decryption as:

$$\text{RLWE}_{\mathbf{s},Q}^{-1}(\mathbf{a}, \mathbf{b}) = \left\lfloor \frac{t}{Q} \cdot [\mathbf{b} - \mathbf{a} \cdot \mathbf{s}]_Q \right\rfloor_t$$

3) *RGSW Ciphertexts*: The Ring-GSW (RGSW) ciphertext is constructed using RLWE ciphertexts and a gadget matrix G . The general definition is:

$$\text{RGSW}_{\mathbf{s}}(m) = (\text{RLWE}'_{\mathbf{s}}(-\mathbf{s} \cdot m), \text{RLWE}'_{\mathbf{s}}(m)),$$

where:

- $\text{RLWE}'_{\mathbf{s}}(m)$ is an extended RLWE ciphertext defined as:

$$\text{RLWE}'_{\mathbf{s}}(m) = (\text{RLWE}_{\mathbf{s}}(m), \dots, \text{RLWE}_{\mathbf{s}}(B^{k-1}m)),$$

with $k = \log_B Q$, where B is the base used for decomposition.

- The gadget matrix G is used for bit decomposition of elements during homomorphic operations.

The RGSW ciphertext supports efficient homomorphic multiplication via an external product operation.

C. Functional Bootstrapping

Functional bootstrapping in TFHE [23] enables the evaluation of arbitrary functions on encrypted data while preserving the security of the underlying plaintext. This process not only evaluates a function f but also reduces the noise in the ciphertext, thereby allowing further homomorphic operations. The workflow is structured into several key transformations, which are described below and summarized in Figure 1.

The functional bootstrapping process involves the following transformations:

- 1) **Initialization**: The accumulator ACC is initialized in the polynomial ring $\mathcal{R}_Q^2$ as:

$$\text{ACC} = (0, X^b \cdot \text{tv}) \in \mathcal{R}_Q^2,$$

where tv is a test vector embedding the Look-Up Table (LUT) representation of the target function f . The term X^b represents a rotation in the ring, with b being derived from the input ciphertext ct .

- 2) **Blind Rotation**: Blind rotation adjusts the accumulator ACC based on the input LWE ciphertext $\text{ct} \in \text{LWE}_{\mathbf{s},q}(m)$ and the bootstrapping key $\mathbf{BSK}$. This operation applies a sequence of homomorphic transformations to ACC:

$$\text{ct}_B \leftarrow \text{BlindRotate}(\text{ACC}, \text{ct}, \mathbf{BSK}, \text{tv}),$$

resulting in $\text{ct}_B \in \text{RLWE}_{\mathbf{s}',Q}(f(m))$. The function f is evaluated during this step, and the output is represented as an RLWE ciphertext.

- 3) **Sample Extraction**: The RLWE ciphertext ct_B is converted back into an LWE ciphertext by extracting a single sample from the polynomial representation:

$$\text{ct}_E \leftarrow \text{SampleExtract}(\text{ct}_B),$$

yielding $\text{ct}_E \in \text{LWE}_{\mathbf{s}',Q}(f(m))$. This step is necessary to transition to the LWE domain for subsequent transformations.

- 4) **Modulus Switching and Key Switching**: These steps involve transitioning the ciphertext modulus and secret key while maintaining the integrity of the encrypted message:

- a) *Modulus Switching*: Given an LWE ciphertext $(\mathbf{a}, b) \in \text{LWE}_{\mathbf{s},q}(m)$, the modulus switching operation computes a new ciphertext $(\mathbf{a}', b') \in \text{LWE}_{\mathbf{s},q'}(m)$, where $q' < q$. The transformation is defined as:

$$a' \leftarrow \lfloor a \cdot \frac{q'}{q} \rfloor, \quad b' \leftarrow \lfloor b \cdot \frac{q'}{q} \rfloor.$$

The modulus of the ciphertext is reduced from Q to a temporary modulus Q_{ks} :

$$\text{ct}_M \leftarrow \text{ModSwitch}(\text{ct}_E, Q, Q_{ks}),$$

where the noise is scaled appropriately to remain within bounds.

- b) *Key Switching*: The secret key of the ciphertext is switched from $\mathbf{s}'$ to $\mathbf{s}$ using the key-switching key $\mathbf{KSK}_{\mathbf{s}' \rightarrow \mathbf{s}}$:

$$\text{ct}_K \leftarrow \text{KeySwitch}(\text{ct}_M, \mathbf{KSK}_{\mathbf{s}' \rightarrow \mathbf{s}}),$$

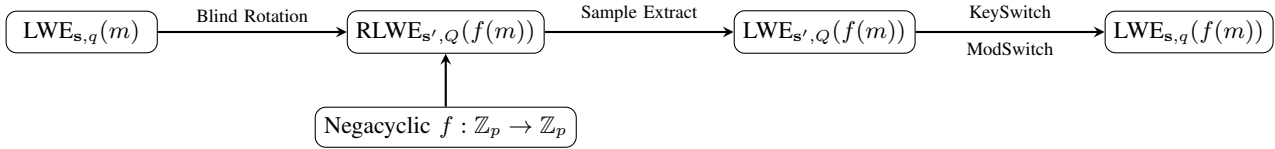


Fig. 1: Functional bootstrapping workflow.

resulting in $ct_K \in \text{LWE}_{s,Q_{ks}}(f(m))$.

- c) *Modulus Switching*: The modulus is further reduced from Q_{ks} to the original modulus q :

$$ct' \leftarrow \text{ModSwitch}(ct_K, Q_{ks}, q),$$

producing the final ciphertext $ct' \in \text{LWE}_{s,q}(f(m))$.

Multi-Value Bootstrapping, as introduced by Carpo et al. (2019) [36], is a technique designed to efficiently evaluate multiple LUTs on the same encrypted input message. This is achieved using essentially a single primary bootstrapping operation followed by less expensive, function-specific finalization steps. The technique is particularly advantageous for Full Domain Functional Bootstrapping algorithms where multiple LUTs often need to be processed concurrently, for instance, Type-SelectMSB requires the evaluation of both positive and negative LUTs. However, applying multi-value bootstrapping can introduce additional noise, with the noise in the output ciphertext ct' being influenced by factors such as the norm of the second-stage polynomial $|TV^{(1)}F_j|_2$. This may lead to decryption failure if not carefully managed (noise gain is detailed in [29]).

III. SECURED EVALUATION OF NON-LINEAR FUNCTIONS

By design, FHE schemes inherently support only homomorphic additions and multiplications. This limitation poses significant challenges when attempting to homomorphically evaluate non-linear functions, such as activation functions used in neural networks. These functions typically involve operations that go beyond the basic arithmetic supported by FHE, necessitating alternative approaches to enable their evaluation.

In this section, we examine in-depth various methods proposed in prior research for the evaluation of non-linear functions under FHE constraints. These methods include polynomial approximation, bitwise computations, and function discretization. We analyze the trade-offs between accuracy and computational efficiency inherent to these approaches, as they strive to balance precision with practical performance.

A. Polynomial Approximation

Early approaches, such as those in CryptoNets [1], used simple square functions for activations like ReLU and sigmoid, while others explored degree-4 polynomials [37] or higher degrees [38] for increased flexibility.

More advanced techniques, like those in CryptoDL [39], employed min-max approximation principles to achieve better accuracy for smooth functions. This direction was further refined by [40]. However, standard min-max methods struggle

with discontinuous functions, necessitating specialized constructions, such as those developed for step functions [41].

Polynomial approximations serve as an effective approach for constructing FHE-compatible representations of non-linear functions, ensuring that the approximated functions remain computationally efficient for homomorphic operations. Among the various techniques available for deriving these approximations, Taylor-series-expansion-based methods are particularly advantageous due to their generality and adaptability across a wide range of activation functions. This universality makes them an appealing choice for approximating non-linear functions in privacy-preserving neural network computations. Below are examples of Taylor series approximations for several commonly used activation functions:

Sigmoid: $\frac{1}{2} + \frac{x}{4} - \frac{x^3}{48} + \frac{x^5}{480} - \mathcal{O}(x^7),$

SiLU: $\frac{x}{2} + \frac{x^2}{4} - \frac{x^4}{48} + \frac{x^6}{480} - \mathcal{O}(x^8),$

Tanh: $x - \frac{x^3}{3} + \frac{2x^5}{15} - \mathcal{O}(x^7),$

GELU: $\frac{x}{2} + \frac{1}{\sqrt{2\pi}} \left(x^2 - \frac{x^4}{6} \right) + \mathcal{O}(x^6),$

ReLU: $\ln(2) + \frac{x}{2} + \frac{x^2}{8} - \mathcal{O}(x^4),$

LeakyReLU: $(1 - \alpha) \ln(2) + \frac{1 + \alpha}{2} x + \frac{1 - \alpha}{8} x^2 - \mathcal{O}(x^4),$

SELU: $\lambda \alpha \left(\frac{x}{2} + \frac{x^2}{8} \right) - \mathcal{O}(x^4).$

B. Domain-Specific Approaches

Various secure inference frameworks employ tailored cryptographic techniques to evaluate non-linear functions efficiently. Some frameworks, such as HEAX, Delphi, and Cheetah [42]–[44], utilize Multi-Party Computation (MPC) for secure evaluation of non-linearities, while others like GAZELLE [45] rely on GC for operations such as ReLU. Swift [11] introduces a hybrid approach by combining FHE and SS, securely comparing plaintext values with zero for ReLU evaluation. Additionally, some frameworks adopt FHE-friendly and application-aware training methods, replacing non-linear functions with approximate alternatives during training to reduce computational complexity and enable efficient inference.

However, none of these approaches provides a comprehensive solution for evaluating arbitrary non-linear functions in FHE schemes. The need for efficient, secure, and flexible methods for evaluating non-linear functions remains a critical area of research.

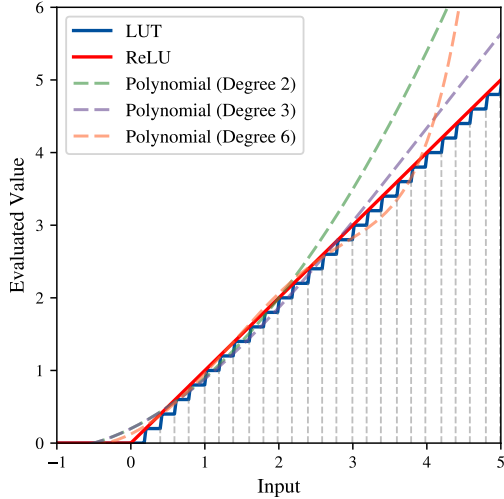


Fig. 2: Visualization of the polynomial approximation method and the LUT-based approach.

C. Functional Bootstrapping

Functional bootstrapping, as described in Section II-C allows for the evaluation of arbitrary functions during the noise-reduction bootstrapping process. This is achieved by embedding a Look-Up Table (LUT) within the bootstrapping operation. A major limitation of standard functional bootstrapping in power-of-2 cyclotomic rings is the “negacyclicity constraint”. The function $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$ must satisfy $f(x + p/2) = -f(x)$. This prevents the direct evaluation of arbitrary functions. Therefore, the function domain was restricted to half of the plaintext space $[0, p/2)$.

Full Domain Functional Bootstrapping (FDFB) was proposed by Klucznik and Schild in 2021 [46], to enable the evaluation of arbitrary functions across the full plaintext domain $\mathbb{Z}_p$. Based on the categorization proposed by Ma et al. [29], FDFB algorithms are classified into three main types:

- **Type-HalfRange** transforms the encrypted message such that its value is constrained to half the plaintext range, effectively bypassing the negacyclic constraints of functional bootstrapping. Algorithms following this design include [47]–[49], with optimizations such as FDFB-Compress [29], which achieves theoretical optimality in this category.
- **Type-SelectMSB** utilizes the most significant bit (MSB) of the encrypted message to switch between two negacyclic LUTs, simplifying the functional bootstrapping process while preserving precision. Contributions to this category include the works of Chillotti et al. [28] and Klucznik and Schild [46].
- **Type-Split** decomposes the LUT into a *pseudo-odd* component and a *pseudo-even* component. Each component is evaluated separately using functional bootstrapping. This approach is used in *Comp* [50].

D. Discussion

This section compares the polynomial-based approach and the LUT-based approach for evaluating non-linear functions within FHE schemes. The analysis focuses on accuracy, computational workload, noise management, and the universality of each method.

Figure 2 visually illustrates the differences between these two approaches. Polynomial-based approximations offer a smooth representation of the target function over a limited domain. However, their effectiveness diminishes as the input deviates from the optimized range. On the other hand, the LUT-based approach provides consistent precision across the entire domain $[0, p)$, offering a more stable alternative for function evaluation.

Accuracy:

- **Polynomial-Based Approach:** This method achieves high accuracy for functions like ReLU, but only within a small, optimized input range. As inputs deviate from this range, the approximation error increases significantly. This makes it less reliable for applications with diverse or unpredictable input distributions.
- **FDFB:** Implemented via functional bootstrapping, this method maintains consistent accuracy across the entire plaintext domain (e.g., $[0, p)$). Because the function is pre-computed and stored in a look-up table, its precision is independent of the input value. This makes LUT-based methods more versatile and reliable, especially for functions with complex input ranges, as illustrated in Figure 2.

Universality and Flexibility:

- **Polynomial-Based Approach:** The universality of polynomials is limited. While effective for simple activations like the square function used to approximate ReLU [51], they struggle to accurately represent complex, non-polynomial functions such as GELU and SiLU, which are critical in modern deep learning models. Furthermore, polynomials are inherently unsuitable for approximating functions with discontinuities or sharp transitions.
- **FDFB:** This method is highly universal. Any arbitrary unary function can be encoded into a Look-Up Table and evaluated during functional bootstrapping. This flexibility allows for the precise implementation of any activation function, from simple to complex, without requiring specialized or custom designs.

Computational Workload and Efficiency:

- **Polynomial-Based Approach:** The computational workload increases with the degree of the polynomial. In word-wise schemes like CKKS, each ciphertext multiplication requires a costly key-switching operation (involving an NTT over a large polynomial ring $\mathcal{R}_{PQ}$). To manage this overhead, prior works often restricted approximations to a low degree, such as degree-2 [1].
- **FDFB:** In FHE schemes like TFHE, the computational cost is primarily determined by the functional bootstrapping operation. This cost is constant regardless of the complexity of the function being evaluated. As detailed in Table II, this approach is often more efficient than

TABLE II: Comparison of polynomial approximations (degree 2, 6, and 9) and functional bootstrapping in terms of noise growth and computational workload.

Method	Degree	Noise Growth	Computational Workload	
			NTT	Inner Product
Polynomial Approximation	2	$2(\sigma_{mul} + \sigma_{ks})$	$2O(PQ n + Q n)$	$2O(n)$
	6	$6(\sigma_{mul} + \sigma_{ks})$	$6O(PQ n + Q n)$	$6O(n)$
	9	$9(\sigma_{mul} + \sigma_{ks})$	$9O(PQ n + Q n)$	$9O(n)$
Functional Bootstrapping		Refresh to Base Level	$(2N + 2B_g N)$ per Blind Rotation	$(2B_g N)$ per Blind Rotation

Note: n is the LWE ciphertext dimension (typically 2^{13} – 2^{17}); N is the RLWE ciphertext dimension (typically around 2^{10}). The bit-length of PQ ranges from 400 to 2600, while B_g is typically 32 bits.

evaluating high-degree polynomials, especially when considering the associated bootstrapping costs in word-wise schemes.

Noise Management:

- **Polynomial-Based Approach:** Homomorphic multiplications are the primary source of noise growth in FHE. Evaluating a high-degree polynomial involves numerous multiplications, leading to rapid noise accumulation. This necessitates frequent and computationally expensive bootstrapping operations to refresh the ciphertext and prevent decryption errors.
- **FDFB:** This is a key advantage of this method. Functional bootstrapping's dual purpose is to evaluate a function *and* simultaneously reset the noise in the ciphertext. This inherent noise-management capability is invaluable in deep computations like neural network inference, as it ensures correctness across layers without incurring the cost of separate, dedicated bootstrapping steps.

While the polynomial-based approach is a practical option for simpler functions or in scenarios with limited computational resources, its limitations in accuracy, universality, and noise management are significant.

In contrast, **FDFB offers a superior solution** for modern deep learning applications. It provides consistent accuracy across the full domain, supports any arbitrary function, and integrates noise management directly into the computation. These features address the primary bottlenecks of FHE, making the LUT-based method a more efficient, robust, and scalable choice for implementing secure neural network inference.

IV. GPU ACCELERATED FDFB FOR NON-LINEAR FUNCTIONS

Graphics Processing Units (GPUs) offer a compelling solution for accelerating such tasks due to their massively parallel architecture. In GPU computing, the *host* (CPU and its memory) manages the overall process and offloads computationally intensive parts to the *device* (GPU and its memory). These intensive parts are coded as *kernels*, which are functions executed by a multitude of *threads* on the GPU. Threads are organized into blocks and grids, allowing for fine-grained parallelism. CUDA streams further enhance efficiency by enabling sequences of operations, such as kernel executions and memory transfers between host and device, to run concurrently, thereby overlapping computation with data movement to hide latency. In this section, we present our implementation of the FDFB framework on GPUs, focusing

Algorithm 1 GPU-Accelerated FDFB-Compress

Input: An LWE Ciphertext $(a, b) = \text{LWE}_{s,q} \left(\frac{q}{p} m' + e \right)$

Input: LUT $F : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$

Output: An LWE ciphertext $\text{LWE}_{s,q} \left(\frac{q}{p} F(m') + e' \right)$

- 1: Set $\text{ACC}_c \leftarrow (0, X^b \cdot \text{tv}_c)$ embedding f_c for compressing
- 2: Set $\text{ACC}_{eval} \leftarrow (0, X^b \cdot \text{tv}_{eval})$ for F
- 3: $\text{ct} \leftarrow (a, b + \frac{q}{2p})$
- 4: $\text{cudaMemcpy } \text{ct} \rightarrow \text{device}$
- 5: $\text{ct} \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_c, \text{ct}, \text{BSK}, \text{tv}_c))$
- 6: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q, Q_{ks})$
- 7: $\text{ct} \leftarrow \text{KeySwitch}(\text{ct}, \text{KSK}_{s' \rightarrow s})$
- 8: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q_{ks}, q)$
- 9: $\text{ct} \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_{eval}, \text{ct}, \text{BSK}, \text{tv}_{eval}))$
- 10: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q, Q_{ks})$
- 11: $\text{ct} \leftarrow \text{KeySwitch}(\text{ct}, \text{KSK}_{s' \rightarrow s})$
- 12: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q_{ks}, q)$
- 13: $\text{cudaMemcpy } \text{ct} \rightarrow \text{host}$

Algorithm 2 GPU-Accelerated FDFB-CancelSign

Input: An LWE Ciphertext $(a, b) = \text{LWE}_{s, \frac{q}{2}} \left(\frac{q}{2p} m' + e \right)$

Input: LUT $F : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$

Output: An LWE ciphertext $\text{LWE}_{s, \frac{q}{2}} \left(\frac{q}{2p} F(m') + e' \right)$

- 1: Set $\text{ACC}_{cs} \leftarrow (0, X^b \cdot \text{tv}_c)$ embedding f_{cs} for cancelsign
- 2: $\text{ct} \leftarrow (a, b + \frac{q}{4p})$
- 3: $\text{cudaMemcpy } \text{ct} \rightarrow \text{device}$
- 4: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, \frac{q}{2}, q)$
- 5: $\text{ct}_1 \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_{cs}, \text{ct}, \text{BSK}, \text{tv}_{cs}))$
- 6: $\text{ct}_{pk} \leftarrow \text{FunctionalKeySwitch}(\text{ct}_1, \text{KSK}_{pk})$
- 7: $\text{ct}_{pk} \leftarrow \text{KeySwitch}(\text{ct}, \text{KSK}_{s' \rightarrow s})$
- 8: Set $\text{ACC}_{pk} \leftarrow (0, X^b \cdot \text{tv}_{pk})$ embedding ct_{pk}
- 9: $\text{ct} \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_{pk}, \text{ct}, \text{BSK}, \text{tv}_{pk}))$
- 10: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q, Q_{ks})$
- 11: $\text{ct} \leftarrow \text{KeySwitch}(\text{ct}, \text{KSK}_{s' \rightarrow s})$
- 12: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q_{ks}, q)$
- 13: $\text{cudaMemcpy } \text{ct} \rightarrow \text{host}$

on the design and optimization strategies employed to enhance performance.

A. Design Overview

To provide a comprehensive implementation, we incorporated algorithms representing distinct categories of FDFB techniques. Specifically, our implementation includes the FDFB-Compress algorithm, which is a Type-HalfRange method, alongside FDFB-CancelSign and FDFB-Select (including its multi-value bootstrapping variant), which are Type-SelectMSB algorithms, as detailed by Ma et al. [29].

The FDFB-Compress algorithm, whose GPU implementation design is described in Algorithm 1 and illustrated in

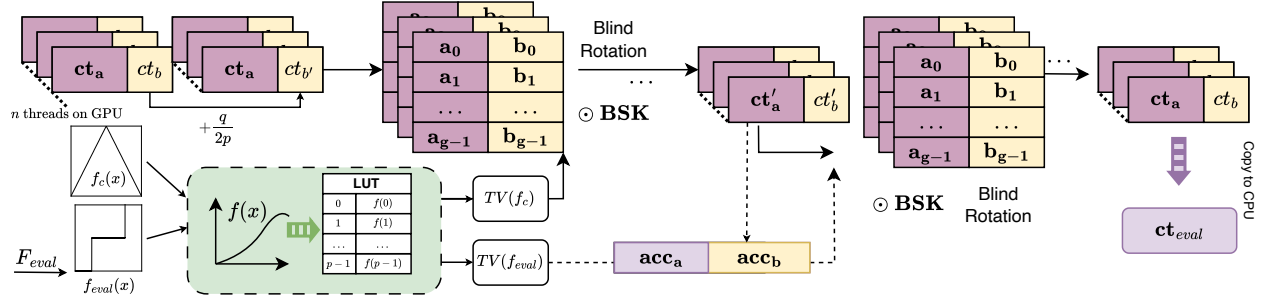


Fig. 3: Implementation design for FDFB-compress.

Algorithm 3 GPU-Accelerated FDFB-Select

Input: An LWE Ciphertext $(a, b) = \text{LWE}_{s,q} \left(\frac{q}{p} m' + e \right)$

Input: LUT: $F : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$

Output: An LWE ciphertext $\text{LWE}_{s,q} \left(\frac{q}{p} F(m') + e' \right)$

- 1: Set $\text{ACC}_{pos} \leftarrow (0, X^b \cdot \text{tv}_{pos})$ for F with MSB 0
 - 2: Set $\text{ACC}_{neg} \leftarrow (0, X^b \cdot \text{tv}_{neg})$ for F with MSB 0
 - 3: Set $\text{ACC}_{sgn} \leftarrow (0, X^b \cdot \text{tv}_{sgn})$ embedding the MSB
 - 4: $\text{ct} \leftarrow (a, b + \frac{q}{2p})$
 - 5: $\text{cudaMemcpy } \text{ct} \rightarrow \text{device}$
 - 6: $\text{ct}_{pos} \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_{pos}, \text{ct}, \text{BSK}, \text{tv}_{pos}))$
 - 7: $\text{ct}_{neg} \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_{neg}, \text{ct}, \text{BSK}, \text{tv}_{neg}))$
 - 8: $\text{ct}_{sgn} \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_{sgn}, \text{ct}, \text{BSK}, \text{tv}_{sgn}))$
 - 9: $\text{ct}_{sgn} \leftarrow \text{ModSwitch}(\text{ct}_{sgn}, Q, Q_{ks})$
 - 10: $\text{ct}_{sgn} \leftarrow \text{KeySwitch}(\text{ct}_{sgn}, \text{KSK}_{s' \rightarrow s})$
 - 11: $\text{ct}_{sgn} \leftarrow \text{ModSwitch}(\text{ct}_{sgn}, Q_{ks}, q)$
 - 12: $\text{ct}_{pk} \leftarrow \text{FunctionalKeySwitch}(\text{ct}_{pos}, \text{KSK}_{pk})$
 $\quad + \text{FunctionalKeySwitch}(-\text{ct}_{neg}, \text{KSK}_{pk}) \cdot X^{\frac{N}{2}}$
 - 13: Set $\text{ACC}_{pk} \leftarrow (0, X^b \cdot \text{tv}_{pk})$ embedding ct_{pk}
 - 14: $\text{ct} \leftarrow \text{SampleExtract}(\text{BlindRotate}(\text{ACC}_{pk}, \text{ct}_{sgn}, \text{BSK}, \text{tv}_{pk}))$
 - 15: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q, Q_{ks})$
 - 16: $\text{ct} \leftarrow \text{KeySwitch}(\text{ct}, \text{KSK}_{s' \rightarrow s})$
 - 17: $\text{ct} \leftarrow \text{ModSwitch}(\text{ct}, Q_{ks}, q)$
 - 18: $\text{cudaMemcpy } \text{ct} \rightarrow \text{host}$
-

Figure 3, consists of four main steps. (a) Precomputation and initialization: The BSK and KSK are loaded into the GPU's global memory, and the input ciphertext is preprocessed to optimize memory usage and prepare it for the subsequent compression step. (b) ciphertext compression: The ciphertext is compressed into the range $[-\frac{q}{4}, \frac{q}{4} - 1]$ by embedding the compression function into an accumulator, performing a blind rotation with the BSK, and applying modulus switching and key-switching to adapt the ciphertext to a smaller modulus. (c) Bootstrapping with LUT for function evaluation: The compressed ciphertext undergoes bootstrapping, during which a target function F encoded in a negacyclic LUT is evaluated. This step includes blind rotations followed by modulus switching and key-switching to maintain compatibility with the reduced modulus and the secret key. (d) Finalization and host transfer: The processed ciphertext is finalized and transferred back to the host. The entire process requires two blind rotations to complete.

In contrast, the FDFB-CancelSign algorithm starts with a ciphertext in $\mathbb{Z}_{\frac{q}{2}}^{(n+1)}$, as detailed in Algorithm 2. The ciphertext is first lifted to $\mathbb{Z}_q^{(n+1)}$, after which it undergoes a transforma-

tion to produce a ciphertext encrypting $(-1)^{\text{MSB}} \left[\frac{Q}{p} F(m') \right]$. Subsequently, an LWE-to-RLWE packing operation, followed by key switching and an additional functional bootstrap, is performed to cancel out the extra $(-1)^{\text{MSB}}$ factor.

Algorithm 3 shows that the FDFB-Select algorithm involves two LUTs, f_{pos} and f_{neg} , which correspond to positive and negative MSBs. A sign function, computed homomorphically on the input ciphertext, determines which of these two LUTs is selected for evaluation. FDFB-Select, in its standard configuration, requires 4 blind rotations. To address scenarios requiring parallel LUT evaluations, we also implemented a multi-value bootstrapping variant of FDFB-Select. This variant permits the concurrent processing of multiple LUTs. However, in our observations, this multi-value bootstrapping approach did not demonstrate superior efficiency compared to the single LUT variant and was found to introduce a significant amount of additional noise. To mitigate these performance characteristics, we explored further optimization of this algorithm through stream-level concurrency, the specifics of which are detailed in Section IV-B.

B. Stream-Level Concurrency

We identified that Algorithm 3 is amenable to stream-level parallelization using CUDA. This approach offers several advantages. Firstly, CUDA streams facilitate the overlapping of Host-to-Device (H2D) and Device-to-Host (D2H) memory transfers with kernel computations, effectively masking the latency associated with these data movements. Secondly, streams can enable the concurrent execution of multiple kernels, which is particularly beneficial when the computational capacity of the GPU is not saturated by a single stream of operations. The architectural design of this parallelized approach is depicted in Figure 4.

It is important to consider the GPU's workload characteristics. When the bootstrapping kernels, especially with a large ciphertext batch size, are sufficient to saturate the GPU's processing capabilities, the degree of parallel execution of kernels across different streams might be constrained. Nevertheless, even in such scenarios, the strategy of overlapping memory transfers with kernel execution remains effective in reducing the overall computation time. Conversely, when the ciphertext batch size is small, individual streams may not fully utilize the GPU's resources. In this situation, the concurrent execution of bootstrapping kernels across multiple streams can yield more

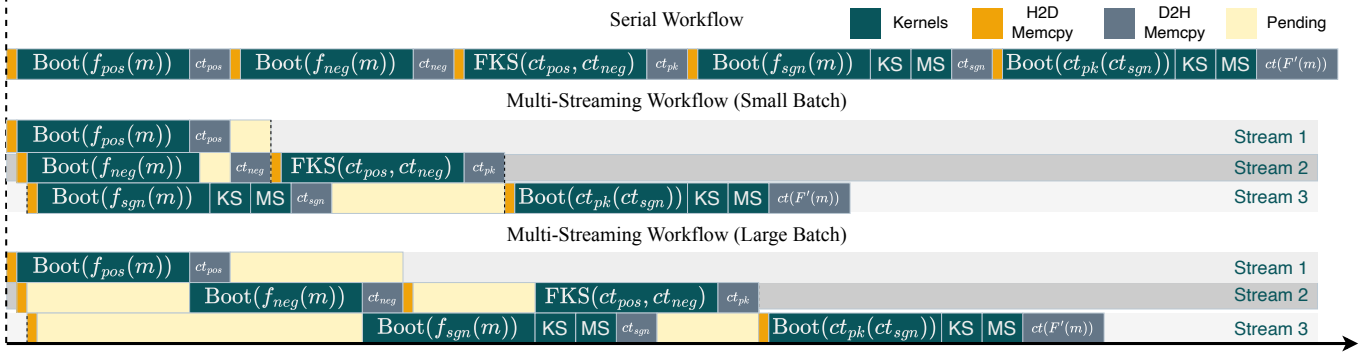


Fig. 4: Stream-Level concurrency design for FDFB-Select. H2D Memcpy: host (CPU) to device (GPU) memory transfer; D2H Memcpy: device to host memory transfer; Kernel: GPU Kernel Execution; Stream: CUDA Stream.

substantial parallel speedups. The performance implications under both of these operational conditions are illustrated in Figure 4.

C. Application-aware Optimization

1) *KSK and BSK Management*: Many FDFB algorithms necessitate distinct key-switching keys (KSK_{pk}) and bootstrapping keys (BSK_g), particularly when employing a small gadget base $B_{g'}$. These cryptographic keys can be substantial in size, potentially consuming a significant portion of, or even exceeding, the available GPU global memory. To address this constraint, we implement a memory-aware management strategy for both KSK and BSK components.

Under this strategy, the complete sets of KSK and BSK are primarily resident in the CPU's main memory (host memory). During the execution of the cryptographic operations on the GPU, only the specific key portions immediately required for the current computational step are transferred to the GPU's global memory (device memory). This on-demand loading mechanism significantly reduces the instantaneous memory footprint on the GPU. By dynamically staging key data, we enable the execution of algorithms that might otherwise be infeasible due to GPU memory limitations and promote more effective utilization of the GPU's computational resources by minimizing memory contention and overallocation.

2) *Ciphertext Batching for Parallel Gate Evaluation*: The plaintext space in TFHE, often represented as a discretized torus $\mathbb{T}_p = \frac{1}{p}\mathbb{Z}/\mathbb{Z}$ where p is typically a small power of 2, results in individual ciphertexts having computational requirements considerably smaller than the processing capacity of modern GPUs. To utilize the available hardware resources more effectively, multiple independent ciphertexts can be aggregated or packed into a single, larger ciphertext structure. This technique, commonly referred to as batching, facilitates the exploitation of data parallelism inherent in GPU architectures. By applying homomorphic operations to these batched ciphertexts, it becomes possible to evaluate numerous logic gates within a circuit simultaneously. For example, the inputs to multiple distinct gates can be processed concurrently using a single batched operation, leading to a substantial increase in computational throughput and enhancing the overall efficiency of evaluating complex circuits homomorphically.

V. EXPERIMENTS AND COMPARISONS

A. Experimental Setup

Our experimental setup utilized a server configured with an Intel Xeon w5-3435X processor and an NVIDIA GeForce RTX 4090 GPU for the primary evaluations. Additionally, supplementary performance assessments were carried out using NVIDIA A100 and RTX 5090 GPUs to further analyze and compare computational efficiencies across different hardware platforms. The experiments were based on the previous GPU acceleration on OpenFHE called VeloFHE [33], which provides a comprehensive framework for implementing functional bootstrapping and other homomorphic encryption operations.

A series of parameters were selected for the experiments, including the plaintext modulus p , the LWE modulus q , the RGSW modulus Q , and the key-switching modulus q_{ks} , as shown in Table III.

B. Performance Evaluation

Table IV presents the per-ciphertext computation time for the FDFB algorithms across different parameter sets and hardware platforms. The results indicate that the GPU implementations significantly outperform the CPU counterparts, with speedups ranging from $80\times$ to $524\times$, depending on the specific algorithm and parameter set used. The FDFB-Compress algorithm demonstrates the most substantial performance improvement, particularly on the RTX 5090 GPU, where it achieves a speedup of $524\times$ compared to the CPU implementation.

Figure 5 demonstrates the runtime efficiency improvements achieved through multi-streaming optimization. The results reveal that the multi-streaming technique significantly reduces the computation time, achieving over 30% reduction in runtime, particularly for smaller batch sizes. As discussed in Section IV-B, smaller batch sizes allow the execution kernels of blind rotations to overlap, leading to more efficient utilization of GPU resources. This overlap enhances concurrency, thereby improving performance.

However, as the batch size increases, the performance gains from multi-streaming gradually diminish. This is attributed to GPU saturation, where the workload of processing large ciphertexts fully utilizes the hardware, leaving limited room

TABLE III: Parameter sets.

Parameter Set	Plaintext	LWE		RLWE /RGSW			Key Switching			
	p	q	n	Q	N	B_g	q_{ks}	B_{ks}	q_{pk}	B_{pk}
#1	16	2^{11}	1340	Q_{53}^*	2^{11}	2^{27}	2^{20}	2^{20}	2^{25}	2^5
#2	32	2^{11}	1340	Q_{53}^*	2^{11}	2^{27}	2^{20}	2^{20}	2^{25}	2^5
#3	16	2^{12}	760	Q_{53}^*	2^{11}	2^{27}	2^{20}	2^{20}	2^{20}	2^5
#4	32	2^{12}	760	Q_{53}^*	2^{11}	2^{27}	2^{20}	2^{20}	2^{20}	2^5

* Q_{53} : a prime number of 53 bits

TABLE IV: Per-ciphertext computation time for FDFB algorithms on A100, 4090, 5090 GPUs, and CPU across parameter sets.

Algorithm	5090 (ms)				4090 (ms)				A100 (ms)				CPU (ms) [29]			
	#1	#2	#3	#4	#1	#2	#3	#4	#1	#2	#3	#4	#1	#2	#3	#4
FDFB-Compress	0.87	0.86	0.54	0.54	1.07	1.07	0.60	0.59	2.07	2.07	1.24	1.24	420.59	419.15	240.14	240.70
FDFB-Select	1.99	2.00	1.32	1.32	2.17	2.17	1.47	1.47	4.19	4.19	3.06	3.08	771.96	774.06	480.55	513.48
FDFB-CancelSign	1.00	0.99	0.68	0.68	1.25	1.28	0.79	0.80	2.15	2.16	1.37	1.37	400.70	390.43	244.10	237.89
FDFB-Select-MultiValue	1.29	0.99	1.36	1.64	1.77	1.79	1.47	1.70	2.62	2.61	2.61	3.30	524.12	520.24	355.94	310.93
Speedup: FDFB-Compress	486×	485×	442×	443×	392×	390×	399×	407×	203×	202×	193×	194×				
Speedup: FDFB-Select	387×	387×	364×	389×	356×	357×	326×	349×	184×	185×	157×	167×				
Speedup: FDFB-CancelSign	401×	393×	361×	350×	321×	305×	309×	296×	186×	181×	178×	174×				
Speedup: FDFB-Select-MultiValue	406×	524×	263×	189×	296×	291×	242×	183×	200×	199×	137×	94×				

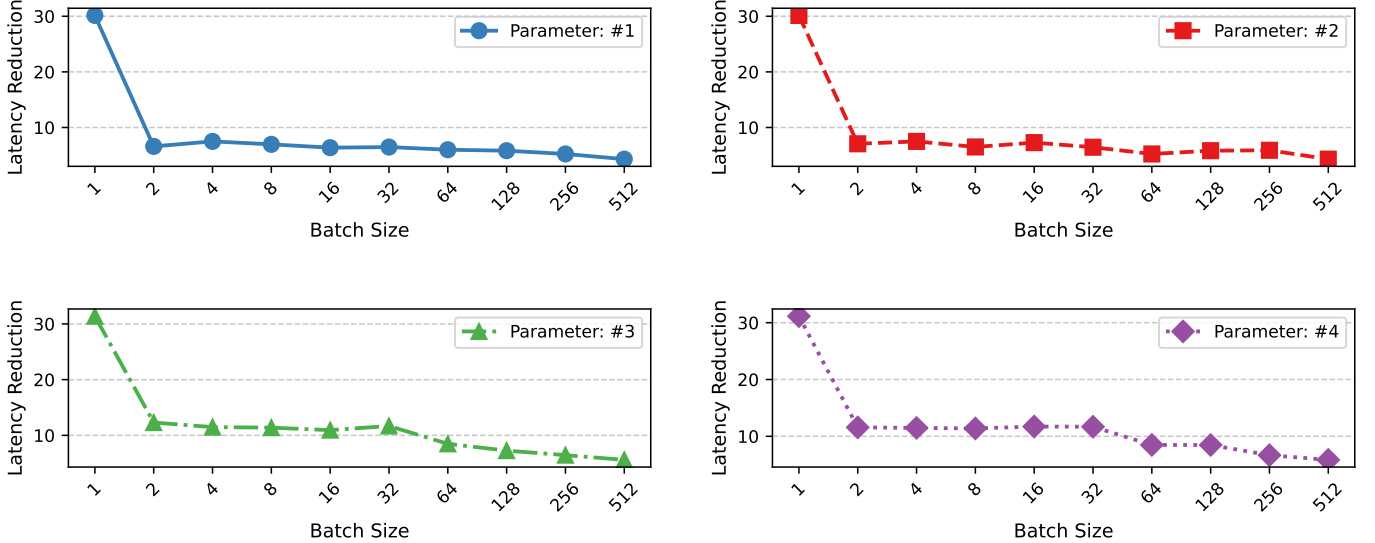


Fig. 5: Per-ciphertext runtime efficiency gains of FDFB-Select with multi-streaming optimization

for additional optimizations. This trend highlights the critical balance between batch size and the effective utilization of CUDA streams to maximize performance in homomorphic encryption tasks.

C. Evaluation on Activation Functions

We evaluated the performance of our GPU-accelerated functional bootstrapping framework using a set of benchmarks. The benchmarks included various activation functions, such as ReLU, Sigmoid, and Tanh, to assess the efficiency and scalability of our implementation.

Figure 6 demonstrates that our approach provides precise and smooth evaluations of ciphertexts across various activation functions over a significant input and output range. This highlights the flexibility and practicality of our method,

making it particularly suitable for inference tasks requiring high-precision evaluations.

Additionally, our method was compared against the polynomial approximation technique described in [52], which employs Chebyshev polynomials to approximate non-linear functions within the CKKS scheme. The comparative results, summarized in Table V, indicate that our implementation achieves superior accuracy, reflected by much lower mean-squared-error (MSE) values compared to the polynomial approximation method. Moreover, the evaluation speed of our GPU-accelerated approach significantly outperforms the polynomial approximation method, particularly when higher-degree polynomials are used. The results show that while the polynomial approximation method requires higher-degree polynomials to achieve comparable evaluation accuracy, this

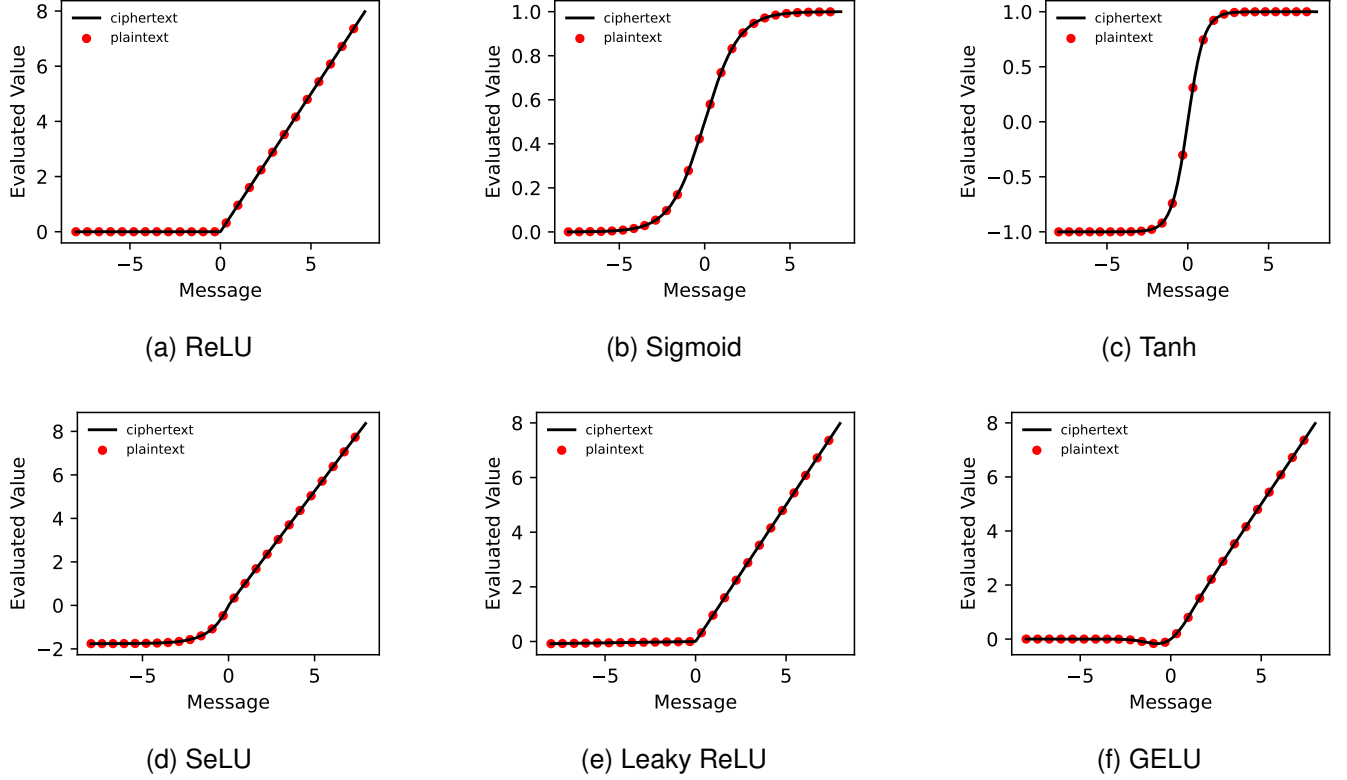


Fig. 6: Homomorphic evaluation results of activation functions: (a) ReLU function. (b) Sigmoid function. (c) Tanh function. (d) SeLU. (e) Leaky ReLU function. (f) GELU function.

TABLE V: Comparison of MSE and throughput (cipher/s) for our approach, degree-2, and degree-6 polynomial approximations.

Activation Function	Mean-Squared-Error (MSE)			Throughput (cipher/s)			Speedup	
	Ours	Degree-2	Degree-6	Ours	Degree-2	Degree-6	vs Degree-2	vs Degree-6
ReLU	6.88×10^{-5}	2.02	0.04	413	76	2	$5.4\times$	$206.5\times$
Sigmoid	5.59×10^{-8}	0.02	0.01	414	24	2	$17.3\times$	$207.0\times$
Tanh	5.90×10^{-8}	0.15	0.03	414	28	2	$14.8\times$	$207.0\times$
SeLU	5.97×10^{-5}	1.83	0.02	415	23	2	$18.0\times$	$207.5\times$
Leaky ReLU	6.26×10^{-5}	1.98	0.04	414	98	2	$4.2\times$	$207.0\times$
GELU	5.80×10^{-5}	2.17	0.05	413	98	2	$4.2\times$	$206.5\times$

increase in degree substantially reduces its evaluation speed, demonstrating the efficiency advantage of our approach.

VI. DISCUSSIONS

Our GPU-accelerated implementation demonstrates significant improvements in performance compared to existing proof-of-concept implementations, particularly for functional bootstrapping of non-linear functions. The results highlight the ability of our approach to fully exploit the parallel processing capabilities of modern GPUs, achieving notable speedups while maintaining high accuracy. This makes the proposed framework a practical solution for real-time applications in homomorphic encryption.

For homomorphic evaluation of non-linear functions, our implementation delivers better performance in both accuracy and speed compared to traditional polynomial approximation methods. This demonstrates the potential of the proposed approach in advancing the capabilities of FHE-based neural

network inference, particularly for non-linear activation functions that are crucial in machine learning tasks.

Restricted performance of Type-SelectMSB algorithms: Despite its theoretical promise, the multi-value variant of the Type-Select algorithm did not deliver significant performance gains in our experiments. This discrepancy arises because the theoretical analysis in [29] assumes that the multi-value bootstrapping method from [36] involves computational overhead equivalent to a single blind rotation. However, in practice, the computational cost of this method is substantially higher due to the dominance of NTT operations and Hadamard products, which exceed the workload of blind rotations. For instance, when evaluating k LUTs simultaneously, the method requires $\lceil \frac{2p}{B_{gmv}} \rceil$ blind rotations to locate the RLWE prime. In addition, there are k digit decomposition and sample extraction operations, which collectively contribute to the overall computational overhead. Even though the multi-value bootstrapping method introduces additional noise, the computational cost is significantly higher than that of performing a single blind

rotation. This inefficiency becomes evident when the number of LUTs is small, as is the case with FDFB-Select, where only three LUTs are required. Consequently, the optimization does not yield substantial performance benefits, and the method does not outperform FDFB-Compress or FDFB-CancelSign.

Nevertheless, Type-Select algorithms demonstrate advantages over Type-HalfRange algorithms in terms of accuracy and flexibility. A notable limitation of Type-HalfRange is the rounding error introduced during the first evaluation LUT, which leads to a loss in accuracy. Furthermore, the parameters of FDFB-Compress are constrained by $p < \frac{q}{4\beta}$, where β is a constant related to bootstrapping noise. These constraints impact the scalability and reliability of Type-HalfRange approaches.

The advancements demonstrated in this work underscore the practical viability of GPU-accelerated FDFB algorithms for FHE-based applications. These improvements are particularly significant for secure and efficient neural network inference, where the accurate evaluation of non-linear activation functions is essential. By addressing the limitations of existing approaches and harnessing GPU parallelism, the proposed framework contributes to the development of scalable and efficient solutions for privacy-preserving computations. Additionally, this work lays the foundation for future exploration, including the integration of GPU-accelerated CKKS with scheme-switching techniques, to further enhance the versatility and applicability of homomorphic encryption in real-world scenarios.

VII. CONCLUSIONS

This work presents a GPU-accelerated implementation of FDFB algorithms for evaluating non-linear functions in FHE, effectively bridging the gap between theoretical advancements and practical applications. The proposed framework enables efficient and precise evaluation of a wide range of activation functions, including complex non-linear functions that are essential for neural network inference.

Moreover, this study highlights limitations in prior methodologies for FHE-secured non-linear function evaluation, including the significant computational overhead caused by bootstrapping and key-switching operations. It also identifies gaps in the analysis of computational overhead in FDFB algorithms, particularly within the GPU context. These insights provide a foundation for further optimizations and refinements of FHE-based solutions in the efficient evaluation of non-linear functions for neural networks.

In conclusion, this work represents a significant step forward in practical FHE-based secure computations. By addressing key performance bottlenecks and leveraging the scalability of GPU-accelerated FDFB algorithms, it contributes to the development of efficient, privacy-preserving solutions for modern machine learning tasks. These advancements pave the way for broader adoption of secure and scalable FHE frameworks in real-world applications.

ACKNOWLEDGEMENT

The authors sincerely acknowledge the financial support provided by the City University of Hong Kong through re-

search grants 9239077 and 9239083, which were instrumental in enabling this study. Additional support was received from the NSFC grant 62372417 and the CCF-Huawei Populus Grove Fund CCF-HuaweiTC202310.

REFERENCES

- [1] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy," in *Proceedings of The 33rd International Conference on Machine Learning*. PMLR, Jun. 2016, pp. 201–210, ISSN: 1938-7228. [Online]. Available: <https://proceedings.mlr.press/v48/gilad-bachrach16.html>
- [2] B. D. Rouhani, M. S. Riazzi, and F. Koushanfar, "Deepsecure: scalable provably-secure deep learning," in *Proceedings of the 55th Annual Design Automation Conference*, ser. DAC '18. New York, NY, USA: Association for Computing Machinery, Jun. 2018, pp. 1–6. [Online]. Available: <https://dl.acm.org/doi/10.1145/3195970.3196023>
- [3] Z. Ge, Z. Zhou, D. Guo, and Q. Li, "Practical Two-party Privacy-preserving Neural Network Based on Secret Sharing," Apr. 2021, arXiv:2104.04709 [cs]. [Online]. Available: <http://arxiv.org/abs/2104.04709>
- [4] T. Ryffel, P. Tholoniati, D. Pointcheval, and F. Bach, "ARIANN: Low-Interaction Privacy-Preserving Deep Learning via Function Secret Sharing," Oct. 2021, arXiv:2006.04593 [cs]. [Online]. Available: <http://arxiv.org/abs/2006.04593>
- [5] P. Yang, Z. L. Jiang, S. Gao, J. Zhuang, H. Wang, J. Fang, S. Yiu, and Y. Wu, "FssNN: Communication-Efficient Secure Neural Network Training via Function Secret Sharing (Full Version)," 2023.
- [6] M. Ball, T. Malkin, and M. Rosulek, "Garbling Gadgets for Boolean and Arithmetic Circuits," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. Vienna Austria: ACM, Oct. 2016, pp. 565–577. [Online]. Available: <https://dl.acm.org/doi/10.1145/2976749.2978410>
- [7] M. Ball, B. Carmer, T. Malkin, M. Rosulek, and N. Schimanski, "Garbled Neural Networks are Practical," 2019, publication info: Preprint. MINOR revision. [Online]. Available: <https://eprint.iacr.org/2019/338>
- [8] Y.-C. Chang and C.-J. Lu, "Oblivious Polynomial Evaluation and Oblivious Neural Learning," in *Advances in Cryptology — ASIACRYPT 2001*, C. Boyd, Ed. Berlin, Heidelberg: Springer, 2001, pp. 369–384.
- [9] M. S. Riazzi, M. Samragh, H. Chen, K. Laine, K. Lauter, and F. Koushanfar, "XONN: XNOR-based Oblivious Deep Neural Network Inference," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1501–1518. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/riazi>
- [10] M. Samragh, S. Hussain, X. Zhang, K. Huang, and F. Koushanfar, "On the Application of Binary Neural Networks in Oblivious Inference," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. Nashville, TN, USA: IEEE, Jun. 2021, pp. 4625–4634. [Online]. Available: <https://ieeexplore.ieee.org/document/9523195/>
- [11] Y. Fu, Y. Tong, Y. Ning, T. Xu, M. Li, J. Lin, and D. Feng, "Swift: Fast Secure Neural Network Inference With Fully Homomorphic Encryption," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 2793–2806, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10908996/>
- [12] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I 23*. Springer, 2017, pp. 409–437.
- [13] A. Viand, P. Jattke, M. Haller, and A. Hithnawi, "HECO: Fully homomorphic encryption compiler," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 4715–4732. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/viand>
- [14] L. Li, J. Lai, P. Yuan, T. Sui, Y. Liu, Q. Zhu, X. Zhang, L. Xiao, W. Chen, and J. Xue, "Ant-ace: An fhe compiler framework for automating neural network inference," in *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, ser. CGO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 193–208. [Online]. Available: <https://doi.org/10.1145/3696443.3708924>

- [15] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, “(leveled) fully homomorphic encryption without bootstrapping,” *ACM Transactions on Computation Theory (TOCT)*, vol. 6, no. 3, pp. 1–36, 2014.
- [16] J. Fan and F. Vercauteren, “Somewhat practical fully homomorphic encryption,” *Cryptology ePrint Archive*, Paper 2012/144, 2012. [Online]. Available: <https://eprint.iacr.org/2012/144>
- [17] Z. Brakerski, “Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP,” in *Advances in Cryptology – CRYPTO 2012*, R. Safavi-Naini and R. Canetti, Eds. Berlin, Heidelberg: Springer, 2012, pp. 868–886.
- [18] J. Fan and F. Vercauteren, “Somewhat practical fully homomorphic encryption,” *Cryptology ePrint Archive*, 2012.
- [19] P. Xie, M. Bilenko, T. Finley, R. Gilad-Bachrach, K. Lauter, and M. Naehrig, “Crypto-Nets: Neural Networks over Encrypted Data,” Dec. 2014, arXiv:1412.6181 [cs]. [Online]. Available: <http://arxiv.org/abs/1412.6181>
- [20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [21] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, R. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [22] L. Ducas and D. Micciancio, “FHEW: Bootstrapping Homomorphic Encryption in less than a second,” in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2015, pp. 617–640.
- [23] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “Faster Packed Homomorphic Operations and Efficient Circuit Bootstrapping for TFHE,” in *Advances in Cryptology – ASIACRYPT 2017*, T. Takagi and T. Peyrin, Eds. Cham: Springer International Publishing, 2017, pp. 377–408.
- [24] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “TFHE: Fast Fully Homomorphic Encryption Over the Torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, Jan. 2020.
- [25] C. Gentry, A. Sahai, and B. Waters, “Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based,” in *Advances in Cryptology – CRYPTO 2013*, R. Canetti and J. A. Garay, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 75–92.
- [26] K. Klucznik and L. Schild, “FDFB: Full Domain Functional Bootstrapping Towards Practical Fully Homomorphic Encryption,” Sep. 2021, arXiv:2109.02731 [cs]. [Online]. Available: <http://arxiv.org/abs/2109.02731>
- [27] K. Klucznik and L. Schild, “FDFB²: Functional bootstrapping via sparse polynomial multiplication,” *Cryptology ePrint Archive*, Paper 2024/1376, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1376>
- [28] I. Chillotti, D. Ligier, J.-B. Orfila, and S. Tap, “Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE,” in *Advances in Cryptology – ASIACRYPT 2021*, M. Tibouchi and H. Wang, Eds. Cham: Springer International Publishing, 2021, vol. 13092, pp. 670–699, series Title: Lecture Notes in Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-3-030-92078-4_23
- [29] S. Ma, T. Huang, A. Wang, Q. Zhou, and X. Wang, “Fast and Accurate: Efficient Full-Domain Functional Bootstrap and Digit Decomposition for Homomorphic Computation,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2024, no. 1, pp. 592–616, Dec. 2023. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/11263>
- [30] N. Gupta, A. Jati, A. K. Chauhan, and A. Chattopadhyay, “PQC Acceleration Using GPUs: FrodoKEM, NewHope, and Kyber,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 3, pp. 575–586, Mar. 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9201530>
- [31] Z. Chen, W. He, J. Wu, J. Liu, B. Xu, X. Bian, and Z. Liu, “An Optimized GPU-based Acceleration of CRYSTALS-Dilithium,” in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2025, pp. 1–5, ISSN: 2379-190X. [Online]. Available: <https://ieeexplore.ieee.org/document/10890327>
- [32] G. Fan, M. Zhang, F. Zheng, S. Fan, T. Zhou, X. Deng, W. Tang, L. Kong, Y. Song, and S. Yan, “Warpdrive: Gpu-based fully homomorphic encryption acceleration leveraging tensor and cuda cores,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1187–1200.
- [33] S. Shen, H. Yang, Z. Liu, Y. Liu, X. Lu, W. Dai, L. Zhou, Y. Zhao, and R. C. C. Cheung, “Velothe: Gpu acceleration for fhe and tfhe bootstrapping,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 3, p. 81–114, Jun. 2025. [Online]. Available: <https://incs.ub.rub.de/index.php/TCHES/article/view/12211>
- [34] Z. Wang, P. Li, R. Hou, Z. Li, J. Cao, X. Wang, and D. Meng, “HE-Booster: An Efficient Polynomial Arithmetic Acceleration on GPUs for Fully Homomorphic Encryption,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1067–1081, Apr. 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10012383>
- [35] F. Zheng, G. Fan, W. Tang, Y. Song, T. Zhou, Y. Zhao, J. Dong, J. Lin, S. Yan, and J. Jing, “GIF-FHE: A Comprehensive Implementation and Evaluation of GPU-Accelerated FHE With Integer and Floating-Point Computing Power,” *IEEE Transactions on Parallel and Distributed Systems*, pp. 1–18, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11016794>
- [36] S. Carpow, M. Izabachène, and V. Mollimard, “New Techniques for Multi-value Input Homomorphic Evaluation and Applications,” in *Topics in Cryptology – CT-RSA 2019*, M. Matsui, Ed. Cham: Springer International Publishing, 2019, vol. 11405, pp. 106–126, series Title: Lecture Notes in Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-3-030-12612-4_6
- [37] H. Chabanne, A. Wargny, J. Milgram, C. Morel, and E. Prouff, “Privacy-preserving classification on deep neural network. cryptology eprint archive,” 2017.
- [38] Q. Zhang, L. T. Yang, and Z. Chen, “Privacy Preserving Deep Computation Model on Cloud for Big Data Feature Learning,” *IEEE Transactions on Computers*, vol. 65, no. 5, pp. 1351–1362, May 2016. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7210184>
- [39] E. Hesamifard, H. Takabi, and M. Ghasemi, “CryptoDL: Deep Neural Networks over Encrypted Data,” Nov. 2017, arXiv:1711.05189 [cs]. [Online]. Available: <http://arxiv.org/abs/1711.05189>
- [40] J. Lee, E. Lee, J.-W. Lee, Y. Kim, Y.-S. Kim, and J.-S. No, “Precise Approximation of Convolutional Neural Networks for Homomorphically Encrypted Data,” *IEEE Access*, vol. 11, pp. 62062–62076, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10155408/>
- [41] T. Huang, S. Ma, A. Wang, and X. Wang, “Approximate Methods for the Computation of Step Functions in Homomorphic Encryption,” in *Information Security and Privacy*, T. Zhu and Y. Li, Eds. Singapore: Springer Nature Singapore, 2024, vol. 14895, pp. 217–237, series Title: Lecture Notes in Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-981-97-5025-2_12
- [42] M. S. Riaz, K. Laine, B. Pelton, and W. Dai, “HEAX: An Architecture for Computing on Encrypted Data,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’20. New York, NY, USA: Association for Computing Machinery, Mar. 2020, pp. 1295–1309. [Online]. Available: <https://dl.acm.org/doi/10.1145/3373376.3378523>
- [43] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, “Delphi: A Cryptographic Inference System for Neural Networks,” in *Proceedings of the 2020 Workshop on Privacy-Preserving Machine Learning in Practice*. Virtual Event USA: ACM, Nov. 2020, pp. 27–30. [Online]. Available: <https://dl.acm.org/doi/10.1145/3411501.3419418>
- [44] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, “Cheetah: Optimizing and Accelerating Homomorphic Encryption for Private Inference,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Feb. 2021, pp. 26–39, ISSN: 2378-203X. [Online]. Available: <https://ieeexplore.ieee.org/document/9407118/>
- [45] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, “GAZELLE: A Low Latency Framework for Secure Neural Network Inference,” in *27th USENIX security symposium (USENIX security 18)*, 2018, pp. 1651–1669. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar>
- [46] K. Klucznik and L. Schild, “FDFB: Full Domain Functional Bootstrapping Towards Practical Fully Homomorphic Encryption,” Sep. 2021, arXiv:2109.02731 [cs]. [Online]. Available: <http://arxiv.org/abs/2109.02731>
- [47] Z. Liu, D. Micciancio, and Y. Polyakov, “Large-Precision Homomorphic Sign Evaluation using FHEW/TFHE Bootstrapping,” 2021, publication info: Published by the IACR in ASIACRYPT 2022. [Online]. Available: <https://eprint.iacr.org/2021/1337>
- [48] A. Guimarães, E. Borin, and D. F. Aranha, “Revisiting the functional bootstrap in TFHE,” *IACR Transactions on Cryptographic Hardware*

and Embedded Systems, pp. 229–253, Feb. 2021. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/8793>

- [49] Z. Yang, X. Xie, H. Shen, S. Chen, and J. Zhou, “TOTA: Fully Homomorphic Encryption with Smaller Parameters and Stronger Security,” 2021, publication info: Preprint. MINOR revision. [Online]. Available: <https://eprint.iacr.org/2021/1347>
- [50] P.-E. Clet, M. Zuber, A. Boudguiga, R. Sirdey, and C. Gouy-Pailler, “Putting up the swiss army knife of homomorphic calculations by means of TFHE functional bootstrapping,” 2022, publication info: Preprint. [Online]. Available: <https://eprint.iacr.org/2022/149>
- [51] R. Livni, S. Shalev-Shwartz, and O. Shamir, “On the computational efficiency of training neural networks,” *Advances in Neural Information Processing Systems*, vol. 1, no. January, pp. 855–863, Dec. 2014, 28th Annual Conference on Neural Information Processing Systems 2014, NIPS 2014 ; Conference date: 08-12-2014 Through 13-12-2014.
- [52] L. Rovida and A. Leporati, “Transformer-based language models and homomorphic encryption: An intersection with BERT-tiny,” in *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, ser. IWSPA ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3–13. [Online]. Available: <https://doi.org/10.1145/3643651.3659893>



Shutong Jin received her B.Eng degree from Department of Electrical Engineering, City University of Hong Kong in 2020. She is now a Ph.D. candidate supervised by Prof. Ray Cheung. Her research interests include privacy-preserving deep learning, GPU acceleration and cryptography.



Donglong Chen received the PhD degree from the Department of Electronic Engineering, City University of Hong Kong, in 2015. He was a visiting research scholar of COSIC, KU Leuven, Belgium, in 2013. After completing his PhD degree study, he spent four years with the industry including Huawei Technology Co., Ltd. and Tencent Technology Co., Ltd. He is currently an associate professor with the Faculty of Science and Technology, Beijing Normal-Hong Kong Baptist University, China. His research interests include cryptographic engineering, software/hardware co-design for AI algorithms, and privacy computing.



Wangchen Dai received the Ph.D. degree in electronic engineering from the City University of Hong Kong in 2018. After that, he had appointments at Hardware Security Lab, Huawei Technologies Company Ltd., the Department of CSSE, Shenzhen University and Zhejiang Lab, Hangzhou, China. He is currently an associate professor at Sun Yat-Sen University. His research interests include cryptographic hardware and embedded systems, fully homomorphic encryption, and reconfigurable computing.



Shiyu Shen received her PhD degree from School of Computer Science, Fudan university in 2024. She is currently a postdoctoral fellow at City University of Hong Kong. Her research interests include lattice-based cryptography, homomorphic encryption, and cryptographic engineering. Her email address is crypto@sheryl.e.dev.



Hao Yang received his PhD degree from College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics in 2024. He is currently a postdoctoral fellow at City University of Hong Kong. His research interests include homomorphic encryption, lattice-based cryptography, and cryptographic engineering. His email address is crypto@d4rk.dev.



Ray C. C. Cheung received the Ph.D. degrees in computing from Imperial College London, London, U.K., in 2007. He conducted his postdoctoral research work with UCLA and his visiting fellowship with Princeton University. He is a Professor with the Department of Electrical Engineering, and an Associate Provost (Digital Learning), CityUHK. His current research interests include cryptographic processor designs and embedded system designs. He is now an Associate Editor of the Journal of Cryptographic Engineering, Springer, and Frontiers in High Performance Computing. He served as the Technical Chair of FPT’02, the General Chair/Co-Chair of ARC’12, FPT’22, and ASAP’24. He is currently the Chair of the IEEE Hong Kong Section.